Ulster University

JavaScript Looping Statements

ulster.ac.uk

Sequential Statements

- Example:

```
1. var x = 2;

2. var y = 3;

3. var z = 0;

4. var z = x + y;

5. Document.write("The sum is :" + z);
```

Ulster University

Loops

- Doing things repeatedly:
  - Loop control – Specifies what condition is necessary for the continued execution of the loop.
  - Loop body – Statements that will be repeatedly executed as long as the loop control condition is satisfied.
  - Three types of looping statements in JavaScript.

Looping Statements

- Very often when you write code, you want the same block of code to run a number of times. You can use looping statements in your code to do this.

- In JavaScript we have the following looping statements:
  - **while** - loops through a block of code while a condition is true.

  - **do...while** - loops through a block of code once, and then repeats the loop while a condition is true.

  - **for** - run statements a specified number of times.

While Statement

- A while statement executes its statements as long as the specified condition is true.

```
while (condition)
  statement
```

- If the condition becomes false, “statement” within the loop stops executing.

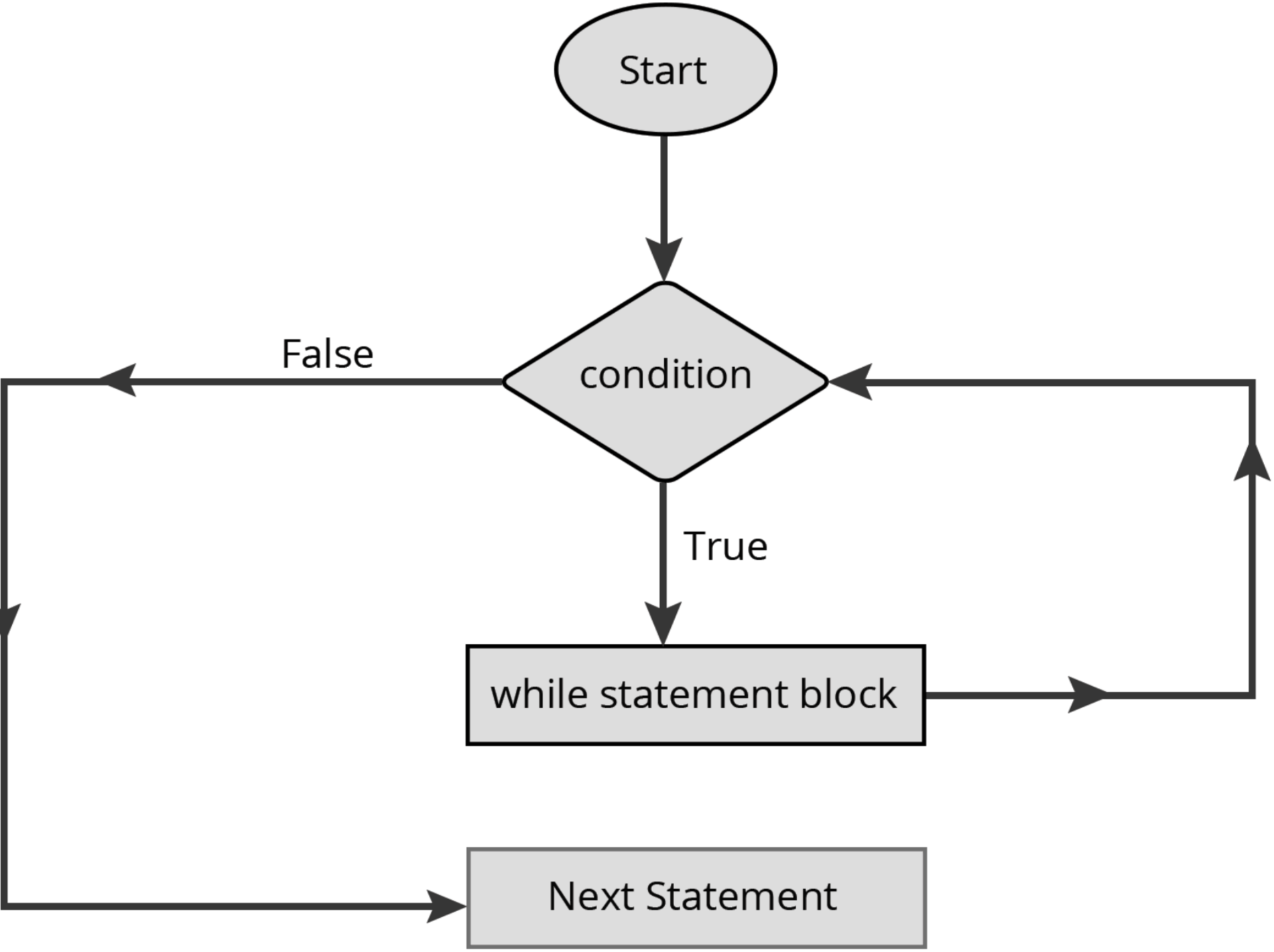
- The condition test occurs before “statement” is executed – if the condition returns “true” then “statement” is executed and the condition is tested again. If the condition returns “false” then the execution stops.

While Loop

- The while loop has the following syntax:

```
while (condition) {
  // code
  // so-called "loop body"
}
```

- While the condition is truthy, the code from the loop body is executed.


Start
condition
False
True
while statement block
Next Statement

While Loop

- For instance, the loop below outputs the value of i while i < 3

```
let i = 0;
while (i < 3) { // shows 0, then 1, then 2
  alert( i );
  i++;
}
```

- A single execution of the loop body is called *an iteration*. The loop in the example above makes three iterations.

- If i++ was missing from the example above, the loop would repeat (in theory) forever.

While Loop

- Any expression or variable can be a loop condition, not just comparisons: the condition is evaluated and converted to a Boolean by while.

- For instance, a shorter way to write while (i != 0) is while (i):

```
let i = 3;
while (i) { // when i becomes 0, the condition becomes falsy
  alert( i );
  i--;
}
```

While Loop

- If the loop body has a single statement, we can omit the curly braces {…}

```
let i = 3;
while (i) alert(i--);
```

While Loop

- The following “while” loop iterates as long as “n” is less than 3.

```
var n = 0;
var x = 0;
while (n < 3) {
  n++;
  x += n;
}
```

- With each iteration the loop increments “n” and adds “x” to that value:
  - After first pass: n=1 and x=1
  - After second pass: n=2 and x=3
  - After third pass: n=3 and x=6

Do…While Loop

- The Do…While statement repeats until a specified condition evaluates to false.

```
do
  statement
while (condition);
```

- However, the “statement” is always executed once before the condition is checked and is only executed thereafter if the condition remains true.

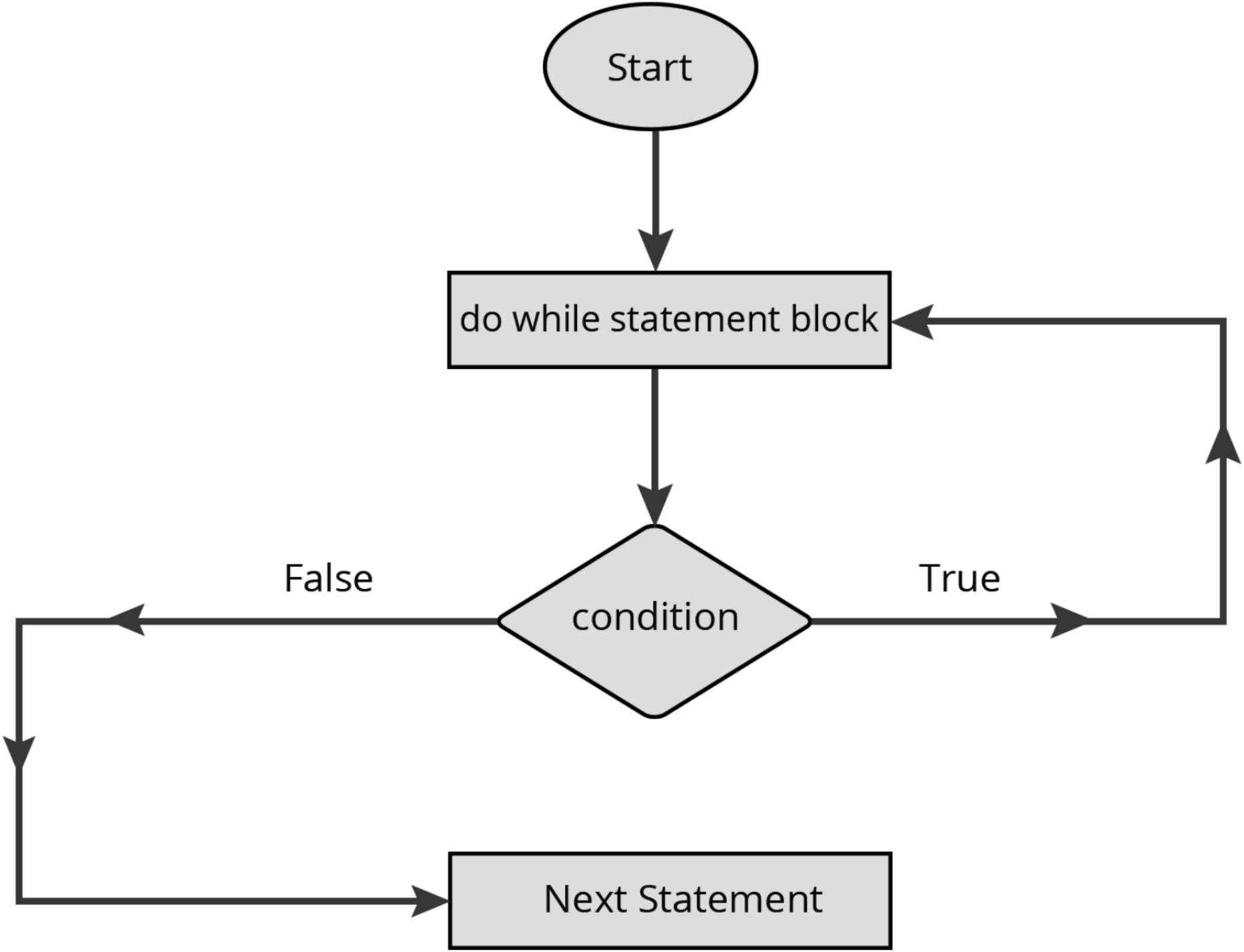
- The condition check can be moved *below* the loop body using the do…while syntax.

Do…While Loop

- The loop will first execute the body, then check the condition, and, while it's truthy, execute it again and again.

```
let i = 0;
do {
  alert( i );
  i++;
} while (i < 3);
```

- The loop will first execute the body, then check the condition, and, while it's truthy, execute it again and again.


Start
do while statement block
False
condition
True
Next Statement

For Loop

- The for loop is more complex, but it’s also the most commonly used loop.

```
for (begin; condition; step) {
  // ... loop body ...
}
```

- It is commonly used to loop through arrays, either to print values out or to search for a specific value within the array.

For Loop

- The loop below runs alert(i) for i from 0 up to (but not including) 3

```
for (let i = 0; i < 3; i++) { // shows 0, then 1, then 2
  alert(i);
}
```

part		
begin	`i = 0`	Executes once upon entering the loop.
condition	`i < 3`	Checked before every loop iteration. If false, the loop stops.
body	`alert(i)`	Runs again and again while the condition is truthy.
step	`i++`	Executes after the body on each iteration.

For Loop

- The general loop algorithm works like this:

```
Run begin
→ (if condition → run body and run step)
→ (if condition → run body and run step)
→ (if condition → run body and run step)
→ ...
```

- That is, begin executes once, and then it iterates: after each condition test, body and step are executed.

- If you are new to loops, it could help to reproduce how it runs step-by-step on a piece of paper.

For Loops

- Here's exactly what happens in our case:

```
// for (let i = 0; i < 3; i++) alert(i)

// run begin
let i = 0
// if condition → run body and run step
if (i < 3) { alert(i); i++ }
// if condition → run body and run step
if (i < 3) { alert(i); i++ }
// if condition → run body and run step
if (i < 3) { alert(i); i++ }
// ...finish, because now i == 3
```

For Loops

- Variable i is declared right in the loop. This is called an “inline” variable declaration. Such variables are visible only inside the loop.

```
for (let i = 0; i < 3; i++) {
  alert(i); // 0, 1, 2
}
alert(i); // error, no such variable
```

- Instead of defining a variable, we could use an existing one:

```
let i = 0;

for (i = 0; i < 3; i++) { // use an existing variable
  alert(i); // 0, 1, 2
}

alert(i); // 3, visible, because declared outside of the loop
```

For Loop

- Any part of for can be skipped. For example, we can omit begin if we don’t need to do anything at the loop start.

```
let i = 0; // we have i already declared and assigned

for (; i < 3; i++) { // no need for "begin"
  alert( i ); // 0, 1, 2
}
```

For Loop

- We can also remove the step part. This makes the loop identical to while (i < 3).

```
let i = 0;

for (; i < 3;) {
  alert( i++ );
}
```

- We can actually remove everything, creating an infinite loop:

```
for (;;) {
  // repeats without limits
}
```

- Please note that the two for semicolons ; must be present. Otherwise, there would be a syntax error.

Breaking a Loop

```
let sum = 0;

while (true) {

  let value = +prompt("Enter a number", '');

  if (!value) break; // (*)

  sum += value;

}
alert( 'Sum: ' + sum );
```

- Normally, a loop exits when its condition becomes falsy.
- But we can force the exit at any time using the special break directive.
- For example, the loop shown asks the user for a series of numbers, “breaking” when no number is entered:

Breaking a Loop

```
let sum = 0;

while (true) {

  let value = +prompt("Enter a number", '');

  if (!value) break; // (*)

  sum += value;

}
alert( 'Sum: ' + sum );
```

- The break directive is activated at the line (*) if the user enters an empty line or cancels the input
- The combination “infinite loop + break as needed” is great for situations when a loop’s condition must be checked not in the beginning or end of the loop, but in the middle or even in several places of its body.

Continuing a Loop

- The continue directive is a “lighter version” of break. It doesn’t stop the whole loop. Instead, it stops the current iteration and forces the loop to start a new one (if the condition allows).

- We can use it if we’re done with the current iteration and would like to move on to the next one. The loop below uses continue to output only odd values:

```
for (let i = 0; i < 10; i++) {

  // if true, skip the remaining part of the body
  if (i % 2 == 0) continue;

  alert(i); // 1, then 3, 5, 7, 9
}
```

Continuing a Loop

- For even values of i, the continue directive stops executing the body and passes control to the next iteration of for (with the next number). So the alert is only called for odd values.

```
for (let i = 0; i < 10; i++) {

  // if true, skip the remaining part of the body
  if (i % 2 == 0) continue;

  alert(i); // 1, then 3, 5, 7, 9
}
```

Continuing a Loop

- The continue directive helps decrease nesting. A loop that shows odd values could look like this:

```
for (let i = 0; i < 10; i++) {

  if (i % 2) {
    alert( i );
  }

}
```

- From a technical point of view, this is identical to the previous example. Surely, we can just wrap the code in an if block instead of using continue.

- But as a side-effect, this created one more level of nesting (the alert call inside the curly braces). If the code inside of if is longer than a few lines, that may decrease the overall readability.

Break/Continue Notes

- Please note that syntax constructs that are not expressions cannot be used with the ternary operator ”?”.

- In particular, directives such as break/continue aren’t allowed there.

```
if (i > 5) {
  alert(i);
} else {
  continue;
}
```

```
(i > 5) ? alert(i) : continue; // continue isn't allowed here
```

- This is just another reason not to use the question mark operator ? instead of if.

Summary

- We covered 3 types of loops:
  - while – The condition is checked before each iteration.
  - do..while – The condition is checked after each iteration.
  - for (;;) – The condition is checked before each iteration, additional settings available.

- To make an “infinite” loop, usually the while(true) construct is used. Such a loop, just like any other, can be stopped with the break directive.

- If we don’t want to do anything in the current iteration and would like to forward to the next one, we can use the continue directive.